APPENDIX C - SDD

FACULTY OF COMPUTING
UTM Johor Bahru

SECJ 3032: Software Engineering PSM1

Semester 01, 2024/2025

Software Design Descriptions (SDD)

Al-Muneer Online Portal

Version 1.0

Prepared by: Ahmed Ghaleb

1. Revision Page

a. Overview

This document is Version 1.0 of the Software Design Description (SDD) for the Al-Muneer Online Portal. It details the high-level and detailed design of the system, building upon the requirements defined in the Software Requirements Specification (SRS) Version 1.0. This SDD describes the system architecture, data design, interface design, and component-level design necessary to guide the implementation of the portal. This document is prepared as part of the SECJ 3032: Software Engineering FYP1 course requirements.

b. Issuing Organisation / Team

- Organisation: Faculty of Computing, Universiti Teknologi Malaysia (UTM)
- Team Name / Developer: Ahmed Hani Ahmed Ghaleb (Individual FYP1 Project)

c. Authorship / Project Team Members

Role	**Team Member**	**Status of Assigned Task (for SDD V1.0)**
Lead Analyst / Designer	Ahmed Hani Ahmed Ghaleb	Complete
Document Author	Ahmed Hani Ahmed Ghaleb	Complete
Technical Architect	Ahmed Hani Ahmed Ghaleb	Complete

d. Revision History

REVISION NO.	**ISSUE DATE**	**DETAILS OF REVISION**
00 (Draft)	15/05/2025	Initial internal draft based on SRS.
1.0	30/05/2025	First complete version of SDD for FYP1 Progress 2.
1.1	8/08/2025	Use case diagram updated: Deleted UC006

Note:
This Software Design Descriptions (SDD template is adapted from IEEE Recommended Practice based on Software Design Descriptions (SDD) (IEEE Std. 10161998 1), that are simplified and customized to meet the need of SECJ3203 FYP1 SE course at Faculty of Computing, UTM. Examples of models are from Arlow and Neustadt (2002) and other sources stated accordingly.

Table of Contents

2. Introduction

The following sections of the Software Design Description (SDD) should provide the details of the entire document.

2.1 Purpose

This Software Design Description (SDD) document delineates the design for the Al-Muneer Online Portal. Its purpose is to translate the functional and non-functional requirements specified in the Software Requirements Specification (SRS) document (Version 1.0) into a detailed blueprint for implementation. This SDD describes the system architecture, data design, component design, interface design, and other design decisions that will guide the development of the portal.

This SDD describes how the Al-Muneer Online Portal will be structured and built. It is intended for use by the project developer (Ahmed Hani Ahmed Ghaleb) as the primary technical guide for system implementation during PSM2, by the project supervisor for reviewing the technical design and ensuring it meets the requirements, and by examiners for assessing the design quality and completeness.

2.2 Scope

This SDD document covers the design of the "Al-Muneer Online Portal," a web-based application for Al-Muneer Hall for Weddings and Events. The scope of the design encompasses:

- **System Architecture:** The overall structure of the system, including its layers, components, and their interactions.

- **Database Design:** The logical and physical design of the relational database (MySQL/PostgreSQL) that will store the portal's data, including entity definitions and relationships.
- **Component Design:** The design of major software components within the backend (Spring Boot) and frontend, detailing their responsibilities and interfaces.
- **Interface Design:**
    - **User Interfaces (UI):** Conceptual design and layout for key user-facing screens for both Visitors and the Administrator.
    - **Internal Interfaces:** APIs between the frontend and backend components.
    - **External Interfaces:** Considerations for any potential external service integrations (e.g., SMS gateway, though detailed integration design is part of implementation).
- **Security Design Considerations:** How security requirements (authentication, authorisation, data protection) will be incorporated into the design.

The software product will, as defined in the SRS:

- Allow visitors to view venue information, media, availability, pricing, submit inquiries, upload payment proofs, and provide feedback.
- Enable administrators to manage all content, inquiries, payment statuses, feedback, view basic reports, and configure notifications.

The design will adhere to the constraints identified in the SRS, including the use of the specified technology stack (Spring Boot, relational database, HTML/CSS/JS), deployment on a Cloud VPS, and considerations for the solo developer context and project timeline. This SDD aims to provide sufficient detail to implement a functional, reliable, and maintainable system as per the project's objectives.

The software product is a custom web application designed specifically for Al-Muneer Hall.

2.3 Context

This SDD document is intended to be used by:
a) The **Project Developer (Ahmed Hani Ahmed Ghaleb)**, who will use this design as the primary guide for the implementation, coding, and unit testing of the Al-Muneer Online Portal during PSM2.
b) The **Project Supervisor (Dr. Muhammad Luqman bin Mohd Shafie)**, who will use this document to review the technical soundness of the design, ensure its alignment with the specified requirements, and assess the progress and quality of the design work.
c) **Academic Examiners**, who will use this document to evaluate the design aspects of the Final Year Project.

This standard (adapted IEEE Std. 1016-1998) is intended for technical and managerial stakeholders who prepare and use SDD. It guides the designer in the selection, organization, and presentation of design information. For an organization (in this case, the academic framework of an FYP) developing its own SDD practices, the use of this standard can help to ensure that design descriptions are complete, concise, consistent, interchangeable, appropriate for recording design experiences and lessons learned, well organized, and easy to communicate.

2.4 Summary

This SDD document provides a comprehensive overview of the design for the Al-Muneer Online Portal.

- **Section 1-4:** Cover introductory material, references, and glossary.
- **Section 5 (Design Body):** This is the core section, detailing the design from multiple viewpoints as per the IEEE standard structure. It includes:
    - Design Stakeholders and Their Concerns (5.1)
    - Context Viewpoint (5.2): System boundary and external interactions (Use Case Diagram).

    - Composition Viewpoint (5.3): High-level system decomposition into major subsystems/modules (Architecture/Component Diagram).
    - Logical Viewpoint (5.4): Detailed class structure within subsystems (Class Diagrams).
    - Information Viewpoint (5.5): Persistent data design (ERD, Data Dictionary).
    - Interface Viewpoint (5.6): User, hardware, and software interface specifications.
    - Structure Viewpoint (5.7): Internal organization of components (Package Diagram).
    - Interaction Viewpoint (5.8): Dynamic behavior between components (Sequence Diagrams for key interactions).
    - State Dynamic Viewpoint (5.9): State changes of key entities (State Machine Diagrams, if applicable at a detailed level).
    - Algorithm Viewpoint (5.10): Logic for complex operations or algorithms (Activity Diagrams for processes).
- **Section 6 (User Interface Design):** Provides conceptual screen layouts and descriptions for key user interfaces.
- **Section 7 (Appendices):** Includes any supporting design information.

This organization aims to present the design in a structured and understandable manner, covering static structures, dynamic behavior, data design, and interface specifications.

3. References

This section provides a complete list of all documents referenced elsewhere in this SDD.

1. Ahmed Hani Ahmed Ghaleb. (2025). *Software Requirements Specification (SRS) - Al-Muneer Online Portal, Version 1.0*. Universiti Teknologi Malaysia. (Internal Project Document)
2. Ahmed Hani Ahmed Ghaleb. (2025). *FYP1 Progress 2 Report - Al-Muneer Online Portal*. Universiti Teknologi Malaysia. (Internal Project Document, containing Chapters 3, 4, 5 of the main report)

3. IEEE. (1998). *IEEE Recommended Practice for Software Design Descriptions* (IEEE Std 1016-1998). Institute of Electrical and Electronics Engineers.
4. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.

Sources for these documents include internal project documentation created as part of the FYP1 process, IEEE standards, established software engineering literature, and official documentation for the chosen technologies.

4. Glossary

This section provides the definitions of all key terms, acronyms, and abbreviations used in this SDD. Many of these are consistent with those defined in the SRS document.

- **Admin:** Administrator of the Al-Muneer Online Portal.
- **API:** Application Programming Interface.
- **Component Diagram:** A diagram that depicts how components are wired together to form larger components or software systems.
- **CRUD:** Create, Read, Update, Delete.
- **ERD:** Entity-Relationship Diagram.
- **FAQ:** Frequently Asked Questions.
- **FYP:** Final Year Project. (Also **PSM**)
- **HTML:** HyperText Markup Language.
- **IDE:** Integrated Development Environment.
- **IEEE:** Institute of Electrical and Electronics Engineers.
- **JS:** JavaScript.
- **MVC:** Model-View-Controller architectural pattern.
- **PaaS:** Platform as a Service.
- **PSM:** Projek Sarjana Muda (Bachelor's Degree Project).
- **REST:** Representational State Transfer – an architectural style for distributed hypermedia systems.
- **SDD:** System Design Document
- **SRS:** Software Requirements Specification.
- **SQL:** Structured Query Language.
- **SSL/TLS:** Secure Sockets Layer/Transport Layer Security.
- **UC:** Use Case.
- **UI:** User Interface.
- **UML:** Unified Modeling Language.
- **UX:** User Experience.
- **VPS:** Virtual Private Server.

5. Design Body

This section of the SDD details the design of the Al-Muneer Online Portal, addressing identified design stakeholders, their concerns, and presenting the design through various viewpoints. The selected design viewpoints are chosen to provide a comprehensive understanding of the system's structure, behavior, and data management, based on the rationale of clarity and completeness for the implementation phase.

5.1 Design Stakeholders and Their Concerns

This section specifies the design stakeholders for the Al-Muneer Online Portal and the primary design concerns of each. An SDD shall address each identified design concern.

a) **The design stakeholders for the design subject (Al-Muneer Online Portal) are:**

* **Mr. Ahmed Almunajid (Venue Owner/Primary User of Admin Panel):** Concerned with the ease of use of the admin panel, the accuracy of information displayed, the efficiency of managing inquiries and bookings, the reliability of the system, and how well it meets his business needs to reduce manual workload and improve customer interaction.

* **Ahmed Hani Ahmed Ghaleb (Student Developer):** Concerned with the clarity of the design for implementation, the feasibility of building the system with the chosen technologies within the given timeframe, the maintainability of the code, adherence to academic requirements, and the overall technical quality of the solution.

* **Prospective Clients/Visitors:** Concerned with the ease of finding information, the clarity and accuracy of content (availability, pricing, venue details), the simplicity of submitting inquiries and payment proofs, and a positive, trustworthy user experience.

* **Project Supervisor & Academic Examiners:** Concerned with the technical soundness of the design, its completeness in addressing the requirements from the SRS, adherence to good software engineering principles, proper documentation, and the overall quality of the project as an academic endeavor.

b) **The design concerns of each identified design stakeholder include:**

* **Mr. Ahmed Almunajid:**

* **Usability:** How easy will it be for him to update content, manage bookings, and use the admin features daily?

* **Functionality:** Will the system reliably perform all the required tasks (calendar updates, inquiry management, payment proof handling, feedback viewing, report generation, notifications)?

* **Data Integrity & Accuracy:** Will the information shown to visitors be accurate and up-to-date? Will booking and payment information be stored reliably?

* **Efficiency:** Will the portal genuinely save time compared to current manual methods?

* **Security (Admin Access):** Is his access to the admin panel secure?

* **Ahmed Hani Ahmed Ghaleb (Developer):**

* **Clarity & Completeness:** Is the design detailed enough to guide implementation without ambiguity?

* **Modularity & Maintainability:** Is the system designed in a way that is easy to code, test, and potentially modify or extend later?

* **Feasibility:** Can all designed aspects be realistically implemented with the chosen technology stack (Spring Boot, MySQL/PostgreSQL, HTML/CSS/JS) and within the project timeline?

* **Performance:** Will the design support the required response times and handle the anticipated load?

* **Security:** Are appropriate security measures designed into the system (input validation, secure data handling, authentication)?

* **Prospective Clients/Visitors:**

* **Ease of Use:** Can they find what they need quickly and intuitively?

* **Information Quality:** Is the information accurate, comprehensive, and clearly presented?

* **Responsiveness:** Does the site load quickly and respond well on different devices?

* **Trust & Reliability:** Does the site appear professional and trustworthy, especially when submitting personal information for inquiries or payment proofs?

* **Accessibility:** Is the site usable across different browsers and devices?

* **Project Supervisor & Academic Examiners:**

* **Requirement Coverage:** Does the design adequately address all functional and non-functional requirements specified in the SRS?

* **Design Soundness:** Are appropriate architectural patterns, data models, and design

principles applied?

* **Documentation Quality:** Is the design well-documented, clear, and consistent?

* **Technical Depth:** Does the design demonstrate a sufficient understanding of software engineering concepts?

* **Adherence to Constraints:** Does the design respect the defined project constraints (technology, timeline, etc.)?

This SDD aims to address these concerns by providing a clear, detailed, and justified design for the Al-Muneer Online Portal.

5.2 Context Viewpoint

The Context viewpoint depicts services provided by a design subject (the Al-Muneer Online Portal) with reference to an explicit context. That context is defined by reference to actors that include users and other stakeholders, which interact with the design subject in its environment. The Context viewpoint provides a “black box” perspective on the design subject.

5.2.1 Design Concerns

This section should specify the following:

a) The purpose of the Context viewpoint is to identify:

- The system (Al-Muneer Online Portal) as a whole.
- The actors that interact with the system (Visitors, Administrator).
- The interactions (services offered/used) between the actors and the system.
- The boundary of the system, distinguishing what is internal and external.

b) The system features from an external "black box" perspective.

The purpose of the Context viewpoint is to identify a design subject's offered services, its actors (users and other interacting stakeholders), to establish the system boundary and to effectively delineate the design subject's scope of use and operation. This is primarily achieved through a Use Case Diagram which illustrates the services (use cases) the system provides to its different actors.

5.2.2 Design View

The system features, from a context viewpoint, are represented by the services it offers to its external actors. These are best illustrated by the Use Case Diagram, which outlines the scope of operations for the Al-Muneer Online Portal.

The system features include:

- For **Visitors**: Viewing venue details, browsing the media gallery, checking availability via an interactive calendar, viewing pricing information, submitting booking inquiries, optionally submitting proof of payment, and providing feedback.
- For the **Administrator**: Securely logging in to manage all website content (hall information, media, availability calendar, pricing, FAQs), managing booking inquiries, managing submitted payment proofs and their statuses, reviewing user feedback, viewing basic operational reports, and configuring system notifications.

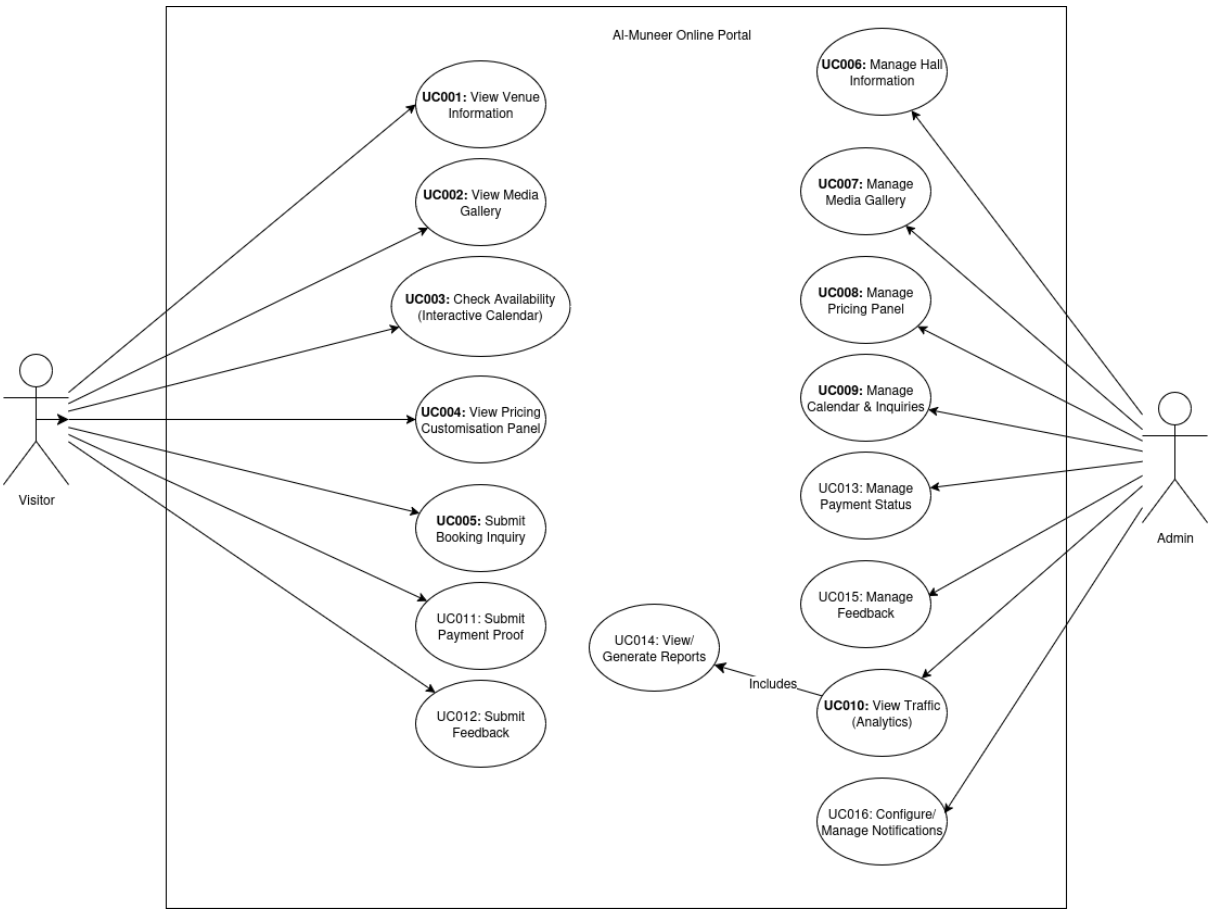


Figure 5.1: Use Case Diagram for Al-Muneer Online Portal

5.3 Composition Viewpoint

The Composition viewpoint describes the way the design subject (Al-Muneer Online Portal) is (recursively) structured into constituent parts and establishes the roles of those parts. It provides a high-level overview of the system's internal organization.

5.3.1 Design Concerns

This should specify the following:

a) **The major design constituents (subsystems/modules):** Identifying the primary high-level blocks that make up the system.

b) **The subsystem responsibilities:** Defining the main role or purpose of each identified subsystem/module.

The purpose of the composition viewpoint is to define the subsystems that compose the system. The Al-Muneer Online Portal is formed by several interacting modules that logically group related functionalities. This decomposition aids in understanding the system's structure, assigning development responsibilities (though a solo project, it helps in organizing work), and managing complexity.

The system formed by these modules aims to deliver the functionalities outlined in the SRS. For the Al-Muneer Online Portal, we can identify the following major logical subsystems:

1. **Presentation Subsystem (Frontend):** Responsible for all user interaction, rendering information, and capturing user input. This subsystem itself can be seen as having two main parts: the Public-Facing Interface and the Administrator Panel Interface.
2. **Application Logic Subsystem (Backend Core):** Responsible for implementing the core business logic, processing requests from the frontend, interacting with the data layer, and managing application state.
3. **Data Management Subsystem (Backend Persistence):** Responsible for all aspects of data storage and retrieval, ensuring data integrity and consistency.
4. **Notification Subsystem (Backend Service):** Responsible for generating and dispatching notifications to users and administrators.
5. **Reporting Subsystem (Backend Service):** Responsible for generating basic operational reports for the administrator.

5.3.2 Design View

This section develops a component model and explains the relationships between the components to achieve the complete functionality of the system. This is a high-level

overview of how responsibilities of the system were partitioned and then assigned to subsystems.

The Al-Muneer Online Portal is decomposed into the following major subsystems/components, which collaborate to deliver the required functionalities. Figure 5.2 provides a high-level component diagram illustrating these major subsystems and their primary interactions.

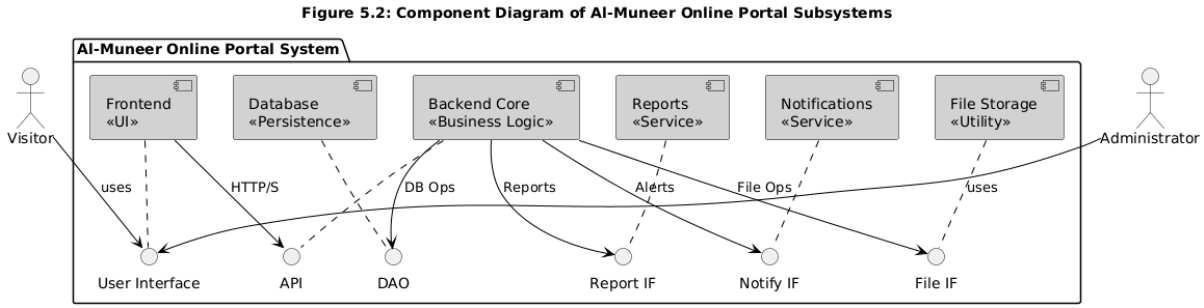


Figure 5.2: Component Diagram of Al-Muneer Online Portal Subsystems

5.4 Logical Viewpoint

The Logical Viewpoint elaborates on existing and designed types and their implementations as classes and interfaces, along with their structural static relationships. This viewpoint provides a detailed look at the internal structure of the software components identified in the Composition Viewpoint, particularly focusing on the organization of classes within the Application Logic Subsystem (Backend Core).

5.4.1 Design Concerns

This should specify the following:

a) **The classes and interfaces:** Identifying key classes and interfaces that realize the system's functionality.
b) **The relationships:** Describing the static relationships between these classes and interfaces (e.g., association, aggregation, inheritance).

The purpose of the logical viewpoint is to elaborate the implementation of classes with their relationships. It focuses on the functional requirements of the system by describing the main classes and their responsibilities within the domain. For the Al-Muneer Online Portal, this involves detailing the key entities, controllers, services, and repositories that constitute the backend system.

5.4.2 Design View

This design view includes a class diagram representing key classes within the respective subsystems/packages, particularly focusing on the controller and domain entity classes for the Al-Muneer Online Portal's backend. The Spring Boot framework encourages a layered architecture often comprising Controller, Service, and Repository layers for each primary domain entity.

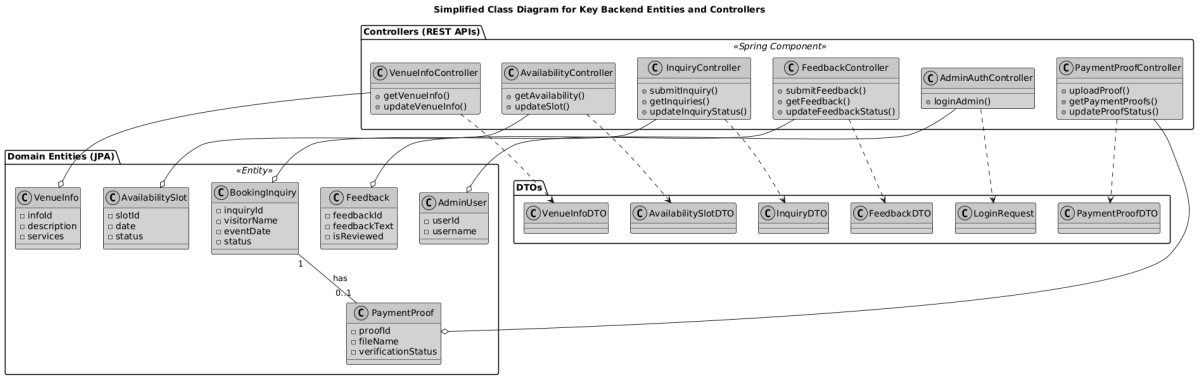


Figure 5.3: Class Diagram for Key Backend Entities and Controllers (Conceptual)

The backend design primarily follows the Model-View-Controller (MVC) pattern, where Spring Boot facilitates this structure.

- **Controller Classes (e.g., VenueInfoController, InquiryController):** These classes are stereotyped as Spring @RestControllers. They handle incoming HTTP requests from the Presentation Subsystem (Frontend), parse request data (often mapped to DTOs), delegate business logic processing to Service classes (not explicitly shown in this simplified diagram but implied), and return HTTP responses (often DTOs converted to JSON). Each controller typically corresponds to a major domain entity or functional area.
- **Domain Entity Classes (e.g., VenueInfo, BookingInquiry, PaymentProof):** These are Plain Old Java Objects (POJOs) typically annotated as JPA @Entitys. They represent the persistent data structures of the application and are mapped to database tables. They contain attributes corresponding to the data fields. Relationships between entities (e.g., a PaymentProof is associated with a BookingInquiry) are defined here.
- **Data Transfer Objects (DTOs) (e.g., InquiryDTO, PaymentProofDTO):** These are simple objects used to transfer data between layers, particularly between the Controller layer and the client (Frontend), and sometimes between the Service and Controller layers. They help decouple the service layer from the presentation layer and can be tailored to specific API needs, preventing exposure of internal entity structures.
- **Service Classes (Implied, not shown in diagram for brevity):** Stereotyped as Spring @Services, these classes contain the core business logic. Controllers delegate calls to them. For instance, an InquiryService would handle the logic for saving a new inquiry, validating its data, and triggering notifications. Services interact with Repository classes.
- **Repository Classes (Implied, not shown in diagram for brevity):** These are interfaces (typically extending Spring Data JPA interfaces like JpaRepository) stereotyped as Spring @Repositorys. They provide an abstraction layer for data access operations (CRUD) on the domain entities, interacting directly with the database.

The relationships shown in the diagram are primarily conceptual associations indicating which controllers manage which entities (via the service and repository layers). The

relationship between BookingInquiry and PaymentProof (one-to-one or one-to-zero/one) is an example of a domain entity relationship. The ERD in the Information Viewpoint (Section 5.5) will provide a more database-centric view of these entity relationships. This logical view ensures a separation of concerns, promotes loose coupling, and enhances the maintainability of the backend system.

5.5 Information Viewpoint

The Information Viewpoint is applicable as there is substantial persistent data content expected with the Al-Muneer Online Portal. Key concerns include persistent data structure, data content, data management strategies, data access schemes, and definition of metadata. This viewpoint describes how the information domain of the system is transformed into data structures and how major data entities are stored, processed, and organized.

5.5.1 Design Concerns

This section explains how the information domain of the Al-Muneer Online Portal is transformed into data structures. It describes how the major data or system entities are stored, processed, and organized. The primary data storage will be a relational database (MySQL/PostgreSQL). This database will consist of several tables, where each table represents an entity and stores its attributes.

The major data or system entities are stored in a relational database named almuneer_portal_db (example name). This database comprises the following key entities, which are detailed further in the Data Dictionary and ERD.

Entity Name List (Conceptual):

1. AdminUser
2. VenueInfo
3. MediaItem
4. AvailabilitySlot
5. PricingTier
6. BookingInquiry
7. PaymentProof
8. Feedback

5.5.2 Design View

This section lists the system entities or major data along with their types and descriptions (attributes), focusing on classes in the domain layer. It also presents the Entity Relationship Diagram (ERD) to show data structure relationships.

Data Dictionary

Entity: AdminUser

Manages administrator login credentials and roles.

Attribute Name	Type	Description
userId	BIGINT	Unique identifier for the user
username	VARCHAR(50)	Login username
passwordHash	VARCHAR(255)	Hashed password
role	VARCHAR(20)	User role (e.g., "ADMIN")

Entity: VenueInfo

Attribute Name	Type	Description
infoId	INT	unique identifier
description	TEXT	Detailed description of the hall
services	TEXT	List of services offered

location	VARCHAR	Physical address of the hall
contactInfo	VARCHAR	Contact details
faqJson	JSON or TEXT	FAQs stored

Entity: MediaItem

Attribute Name	Type	Description
mediaId	BIGINT	Unique identifier for the media item
fileName	VARCHAR	Original name of the uploaded file
filePath	VARCHAR	Optional caption for the media
mediaType	VARCHAR	Optional caption for the media
caption	VARCHAR	Optional caption for the media
uploadDate	TIMESTAMP	Date and time of upload

Entity: AvailabilitySlot

Attribute Name	Type	Description
slotId	BIGINT	Unique identifier for the slot
slotDate	DATE	The specific date
status	VARCHAR	Booking status (e.g., "available", "booked", "pending_confirmation")
notes	TEXT	Internal notes for the admin regarding this slot
updatedAt	TIMESTAMP	Last update timestamp

Entity: PricingTier

Attribute Name	Type	Description
pricingId	BIGINT	Unique identifier for the tier
tierName	VARCHAR	Name of the pricing tier/package
basePrice	DECIMAL	Base price for this tier
description	TEXT	Details of what the tier includes
isActive	BOOLEAN	Whether this tier is currently active

Entity: BookingInquiry

Attribute Name	Type	Description
inquiryId	BIGINT	Unique identifier for the inquiry

visitorName	VARCHAR	Name of the person making the inquiry
visitorContact	VARCHAR	Contact number (phone/WhatsApp)
visitorEmail	VARCHAR	Optional email of the visitor
eventDate	DATE	Desired date for the event
numGuests	INT	Estimated number of guests
eventType	VARCHAR	Type of event (e.g., Wedding, Gathering)
message	TEXT	Specific questions or requirements
submissionDate	TIMESTAMP	Date and time inquiry was submitted
status	VARCHAR(50)	Status (e.g., "new", "viewed", "contacted", "payment_pending", "confirmed", "cancelled")
adminNotes	TEXT	Internal notes by admin for this inquiry

Entity: PaymentProof

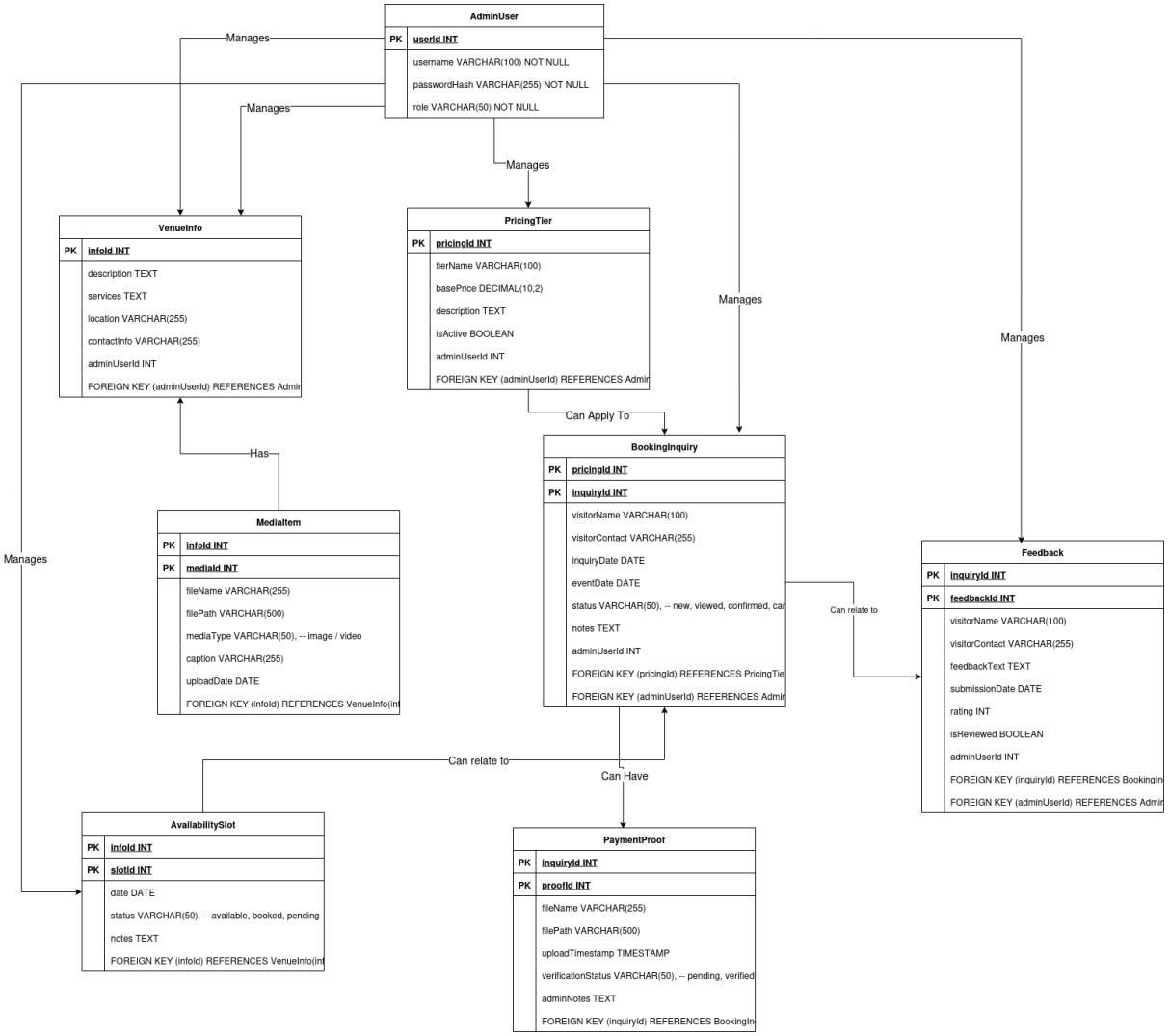
Attribute Name	Type	Description
proofId	BIGINT	Unique identifier for the proof
inquiryId	BIGINT	Foreign key linking to BookingInquiry
fileName	VARCHAR	Original name of the uploaded file
filePath	VARCHAR	Server path where the file is stored
uploadTimestamp	TIMESTAMP	Date and time of upload
verificationStatus	VARCHAR	Status (e.g., "pending", "verified", "rejected")
adminNotes	TEXT	Internal notes by admin for this proof

Entity: Feedback

Attribute Name	Type	Description
feedbackId	BIGINT	Unique identifier for the feedback
visitorName	VARCHAR	Optional name of the person providing feedback
visitorContact	VARCHAR	Optional contact details
feedbackText	TEXT	The feedback message itself
rating	INT	Optional rating (e.g., 1-5 stars)
submissionDate	TIMESTAMP	Date and time feedback was submitted

isReviewed	BOOLEAN	Whether admin has reviewed this feedback

Entity Relationship Diagram (ERD)



5.6 Interface Viewpoint

The Interface Viewpoint provides information for designers, programmers, and testers to correctly use the services provided by the Al-Muneer Online Portal and for the portal to interact with other entities. This description includes details of external and internal interfaces. This viewpoint consists of a set of interface specifications for each entity that interacts with the system or for interfaces between major system components.

5.6.1 Design Concerns

This section describes the functionality of the system from the user's perspective and explains how users will interact with the system to complete expected features. It also covers interfaces with other hardware or software. For the Al-Muneer Online Portal, the interface viewpoint primarily covers User Interfaces (UI) for Visitors and the Administrator, and Software Interfaces (APIs) between the frontend and backend. Direct hardware interfaces are minimal for a web application beyond standard peripheral use.

5.6.2 **Design View**

User Interfaces

This specifies the logical characteristics of each interface between the software product and its users, including configuration characteristics necessary to accomplish the software requirements, and aspects of optimizing the interface.

a) **Logical Characteristics:**

The Al-Muneer Online Portal will feature a web-based graphical user interface (GUI) accessible via standard web browsers.

* **Visitor Interface:**

* **Screen Layouts:** Will consist of clear, well-organized pages for Homepage, Venue Information, Media Gallery, Availability Calendar, Pricing, FAQ, Booking Inquiry, Payment Proof Submission, and Feedback. Navigation will be consistent via a main menu.

* **Content:** Will display textual information, images, videos, an interactive calendar, and forms for user input.

* **Interaction:** Primarily through mouse clicks, touch on touch-enabled devices, and keyboard input for forms.

* **Administrator Interface (Admin Panel):**

* **Screen Layouts:** A secure, dashboard-style interface with sections for managing venue content, availability, pricing, inquiries, payment proofs, feedback, reports, and notification

settings. Will heavily utilize forms, tables for data display, and action buttons.
* **Content:** Will display system data in manageable formats, configuration options, and input fields for updates.
* **Interaction:** Secure login required. Interaction via mouse clicks and keyboard input for data entry and management tasks.

b) **Optimizing the Interface:**
* **Simplicity:** Interfaces will be kept clean and uncluttered, avoiding unnecessary complexity, especially for the admin panel to ensure ease of use for Mr. Almunajid.
* **Consistency:** Navigation, terminology, and layout will be consistent across the portal.
* **Feedback:** The system will provide immediate visual feedback for user actions (e.g., successful form submission, error messages). Error messages will be informative.
* **Efficiency:** Forms will be designed to minimize clicks and data entry effort.
* **Language:** Primarily Arabic, with consideration for English in key navigational elements, aligning with user characteristics.

Hardware Interfaces

This specifies the logical characteristics of each interface between the software product and the hardware components of the system. For the Al-Muneer Online Portal, a web-based application, there are no direct, custom hardware interfaces designed into the software itself. The system will rely on standard user hardware:

- **Client-side:** User's computer (desktop, laptop), tablet, or smartphone with a web browser and internet connection. Standard input devices (keyboard, mouse, touchscreen) are assumed.
- **Server-side:** The Cloud VPS will have standard server hardware (CPU, RAM, storage, network interface) managed by the hosting provider. The application will interact with this hardware through the operating system and the Java Virtual Machine.

Software Interfaces

This specifies the use of other required software products and interfaces with other application systems.

1. **Operating Systems:**
    - **Client-Side:** The portal will be accessible from standard operating systems that support modern web browsers (e.g., Windows, macOS, Linux, Android, iOS).
    - **Server-Side:** The Cloud VPS will run a common server operating system (e.g., a Linux distribution like Ubuntu).
    - **Interface:** Standard interactions via web browser (client) and JVM/OS system calls (server).
2. **Web Server:**
    - The Spring Boot application includes an embedded web server (e.g., Tomcat, Jetty, or Undertow by default).
    - **Interface:** HTTP/HTTPS protocols for communication with client browsers.
3. **Database Management System (DBMS):**
    - MySQL or PostgreSQL.
    - **Interface:** The Spring Boot application will interface with the DBMS using JDBC (Java Database Connectivity) and Spring Data JPA for data persistence and retrieval.
4. **Web Browsers (Client-Side):**
    - Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge (latest stable versions).
    - **Interface:** The system will output HTML, CSS, and JavaScript that these browsers render. It will receive user input via HTTP requests.
5. **SMS Gateway API (Optional External Service):**
    - **Name:** To be determined based on availability, cost, and ease of integration (e.g., Twilio, Vonage, or a local Yemeni provider if an accessible API exists).
    - **Purpose:** To send SMS notifications to users and/or the administrator.
    - **Interface:** If implemented, the backend (Notification Subsystem) will make HTTP API calls to the chosen SMS gateway service, adhering to that service's

API specifications (message content, format, authentication). This interface design is contingent on selecting a specific provider.

6. **Email Service (Optional External Service / Internal Library):**
    - JavaMail API (for sending emails via SMTP if basic email notifications are implemented).
    - **Purpose:** To send email notifications (e.g., inquiry confirmation to admin if SMS is not primary).
    - **Interface:** The backend (Notification Subsystem) will use the JavaMail library to connect to an SMTP server (e.g., Gmail, a hosting provider's SMTP, or a dedicated email service) to send emails. This involves configuring SMTP host, port, authentication.

Communication Interfaces

This specifies the various interfaces to communications such as local network protocols, etc.

- **HTTP/HTTPS:** The primary communication protocol between the client web browser and the web server hosting the Al-Muneer Online Portal. HTTPS will be enforced for secure data transmission.
- **TCP/IP:** The underlying network protocol suite for all internet-based communication.
- **JDBC:** Used for communication between the Java application (Spring Boot backend) and the relational database (MySQL/PostgreSQL).

No other specialized local network protocols are designed for this system beyond these standard web and database communication protocols.

5.7 Structure Viewpoint

The Structure Viewpoint is used to document the internal constituents and organization of the design subject (Al-Muneer Online Portal) in terms of like elements (recursively). It informs the user and the developer about the interaction between the packages in the system and how the software is organized into modules or components at a finer granularity than the Composition Viewpoint.

5.7.1 Design Concerns

The primary design concerns for the Structure Viewpoint are:

- **Modularity:** How the system is broken down into manageable, cohesive, and loosely coupled packages or modules.
- **Dependencies:** The relationships and dependencies between these packages.
- **Organization:** The overall architectural organization of the code within the backend application.
- **Reusability:** Identifying components or packages that might be reusable.
- **Maintainability:** Ensuring the structure supports ease of understanding and modification.

Structure viewpoint informs user and the developer about the interaction between the packages in the system. For the Al-Muneer Online Portal, the backend developed using Spring Boot will be organized into several key packages, each with specific responsibilities, reflecting a layered architecture and separation of concerns.

5.7.2 Design View

This section includes the overall package diagram of the system, primarily focusing on the backend (Spring Boot application) structure. The navigation visibility is based on the dependency among classes in the design class diagram (Logical Viewpoint).

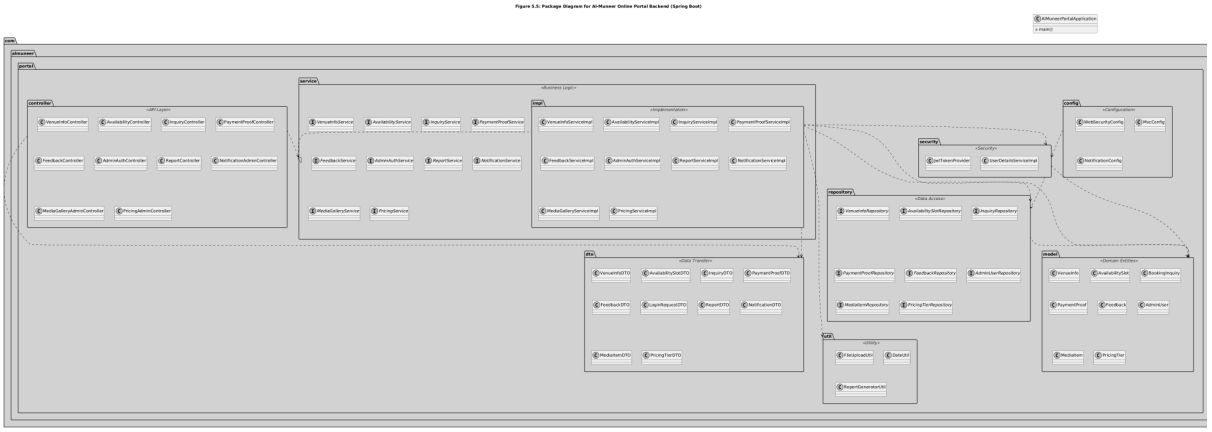


Figure 5.5: Package Diagram for Al-Muneer Online Portal Backend (click on it to expand)

Description of Backend Packages:

The backend of the Al-Muneer Online Portal, built with Spring Boot, is organized into the following primary packages:

- **com.almuneer.portal (Root Package):** Contains the main application class (AlMuneerPortalApplication) that bootstraps the Spring Boot application.
- **config:** This package holds configuration classes for the application.
    - WebSecurityConfig: Configures Spring Security for authentication, authorization, password hashing, and defining protected routes (e.g., for the admin panel).
    - MvcConfig: Contains configurations related to Spring MVC, such as view resolvers (if not using pure REST), static resource handling, or interceptors.
    - NotificationConfig: May contain beans or configurations related to the notification service (e.g., SMS gateway credentials, email server settings).
- **controller (API Layer):** This package contains REST controller classes that handle incoming HTTP requests from the frontend.

    - Each controller (e.g., InquiryController, AdminAuthController) is responsible for a specific domain or functional area. They receive requests, validate them (often delegating complex validation), call appropriate methods in the service layer, and formulate HTTP responses, typically using DTOs.
- **service (Business Logic Layer):** This package contains interfaces defining the business logic contracts and an impl sub-package containing their concrete implementations.
    - Service interfaces (e.g., InquiryService, PaymentProofService) define the business operations.
    - Implementation classes (e.g., InquiryServiceImpl) contain the core business logic, orchestrating calls to repositories, other services, and utility classes to fulfill the use cases. They handle transactions and business rule enforcement.
- **repository (Data Access Layer):** This package contains Spring Data JPA repository interfaces (e.g., InquiryRepository, AdminUserRepository).
    - These interfaces extend Spring Data interfaces (like JpaRepository) to provide CRUD operations and custom query methods for interacting with the database entities (model package) without requiring boilerplate DAO code.
- **model (Domain Entities):** This package contains the Plain Old Java Objects (POJOs) that represent the persistent data entities of the application, as defined in the Information Viewpoint (ERD and Data Dictionary).
    - These classes (e.g., BookingInquiry, AdminUser) are typically annotated with JPA (@Entity) to map them to database tables.
- **dto (Data Transfer Objects):** This package contains classes used to transfer data between layers, especially between the controller and the frontend, and sometimes between service and controller.
    - DTOs (e.g., InquiryDTO, LoginRequestDTO) help decouple the internal domain model from the API structure and can be tailored for specific request/response needs.
- **util (Utility Classes):** This package holds common utility classes that provide helper functions used across different parts of the application.
    - Examples include FileUploadUtil for handling file uploads (like payment proofs), DateUtil for date/time manipulations, or ReportGeneratorUtil for basic report formatting logic.

- **security:** This package contains classes related to application security, primarily for authentication and authorization using Spring Security.
    - JwtTokenProvider (if using JWT for session management) would handle token generation and validation.
    - UserDetailsServiceImpl would implement Spring Security's UserDetailsService to load user-specific data (from AdminUserRepository) for authentication.

This package structure promotes a clear separation of concerns, modularity, and adherence to common practices in Spring Boot application development, making the system more organized and maintainable. Dependencies are managed such that controllers depend on services, services depend on repositories, and repositories interact with domain models. DTOs are used for data exchange, and utility/configuration/security packages provide cross-cutting concerns.

5.8 Interaction Viewpoint

The Interaction Viewpoint defines strategies for interaction among design entities (components, classes, subsystems), regarding why, where, how, and at what level actions occur. It illustrates the dynamic behavior of the system, showing how different parts of the system collaborate to achieve specific functionalities.

5.8.1 Design Concerns

The primary design concerns for the Interaction Viewpoint are:

- **Dynamic Behavior:** How system components and objects interact over time to accomplish tasks.

- **Collaboration:** Understanding the sequence of messages or calls exchanged between different parts of the system.
- **Responsibility Allocation:** Clarifying which component is responsible for initiating an interaction and which component responds.
- **Use Case Realization:** Showing how the interactions realize the use cases defined in the SRS.

The functionalities of the system are given by the help of sequence diagrams. Moreover, it defines strategies for interaction among entities. For the Al-Muneer Online Portal, sequence diagrams are used to model these interactions.

5.8.2 Design View

sequence

Detailed sequence diagrams illustrating the step-by-step interactions for each specific use case (UC001 through UC016), involving actors, frontend components, backend controllers, services, and database interactions, have already been provided in the **Software Requirements Specification (SRS) document, Section 2.3: Use Case Details**. These SRS sequence diagrams provide a granular view of how each functional requirement is realized through object interactions.

This section of the SDD will therefore focus on providing a higher-level view of interactions between the major subsystems identified in the Composition Viewpoint (Section 5.3, Figure 5.2). These diagrams illustrate how these coarse-grained components collaborate for key overarching scenarios.

a) SD001: High-Level Sequence Diagram for Visitor Submitting a Booking Inquiry and Initial Admin Notification

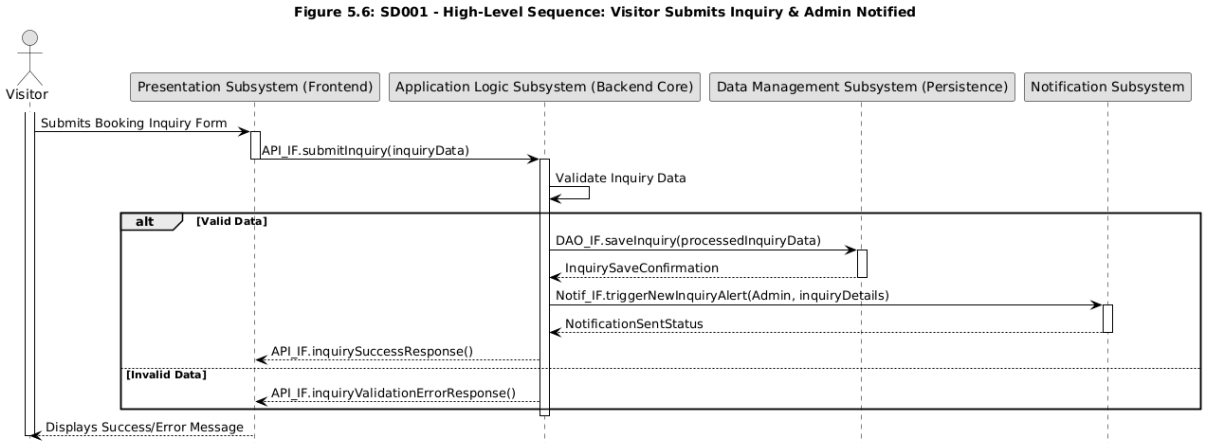


Figure 5.6: SD001 - High-Level Sequence: Visitor Submits Inquiry & Admin Notified

Description of SD001:

The Visitor initiates the process by submitting the booking inquiry form via the Presentation Subsystem (Frontend). The Frontend forwards this request to the Application Logic Subsystem via its API. The Application Logic Subsystem first validates the received data. If valid, it instructs the Data Management Subsystem to save the inquiry. Upon successful saving, the Application Logic Subsystem then triggers the Notification Subsystem to alert the Administrator about the new inquiry. Finally, a response (success or validation error) is sent back to the Frontend, which then updates the Visitor.

b) SD002: High-Level Sequence Diagram for Administrator Managing Payment Proof Status

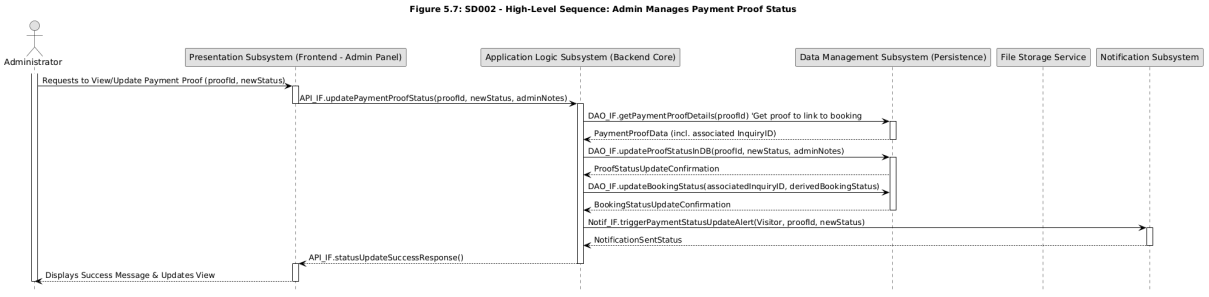


Figure 5.7: SD002 - High-Level Sequence: Admin Manages Payment Proof Status

Description of SD002:

The Administrator, via the Admin Panel (Presentation Subsystem), initiates an update to a payment proof's status. The request is sent to the Application Logic Subsystem. The AppLogic first retrieves details of the payment proof (and potentially the associated booking inquiry) from the Data Management Subsystem. It then instructs the Data Management Subsystem to update the status of the payment proof and possibly the linked booking inquiry. After successful database updates, the AppLogic triggers the Notification Subsystem to inform the Visitor of their payment status update. A success response is then relayed to the Admin Panel, which updates the Administrator's view. The File Storage Service would have been involved earlier when the admin viewed the actual proof image, orchestrated by the AppLogic.

5.9 State Dynamic Viewpoint

Reactive systems and systems whose internal behavior is of interest use this viewpoint. System dynamics including modes, states, transitions, and reactions to events are described here. This section focuses on the behavior and states of key entities within the Al-Muneer Online Portal, illustrated via state transition diagrams. This informs designers, developers, and testers about the dynamic view of critical parts of the system.

5.9.1 Design Concerns

The primary design concerns for the State Dynamic Viewpoint are:

- **Entity Lifecycles:** Understanding the different states an entity can be in throughout its lifecycle.
- **State Transitions:** Identifying the events or conditions that cause an entity to transition from one state to another.
- **Actions/Activities:** Specifying any actions or activities that occur when entering a state, exiting a state, or during a transition.
- **System Behavior:** Clarifying how key entities respond to various system events and user interactions.

In this section, system behavior and states of the system are given via the state transition diagram. Designers, developers and testers are informed about dynamic view of the system which includes its states, transitions, and the events that trigger these transitions for key entities.

5.9.2 Design View

State transition diagrams are included here for key entities in the Al-Muneer Online Portal whose behavior involves distinct and significant state changes.

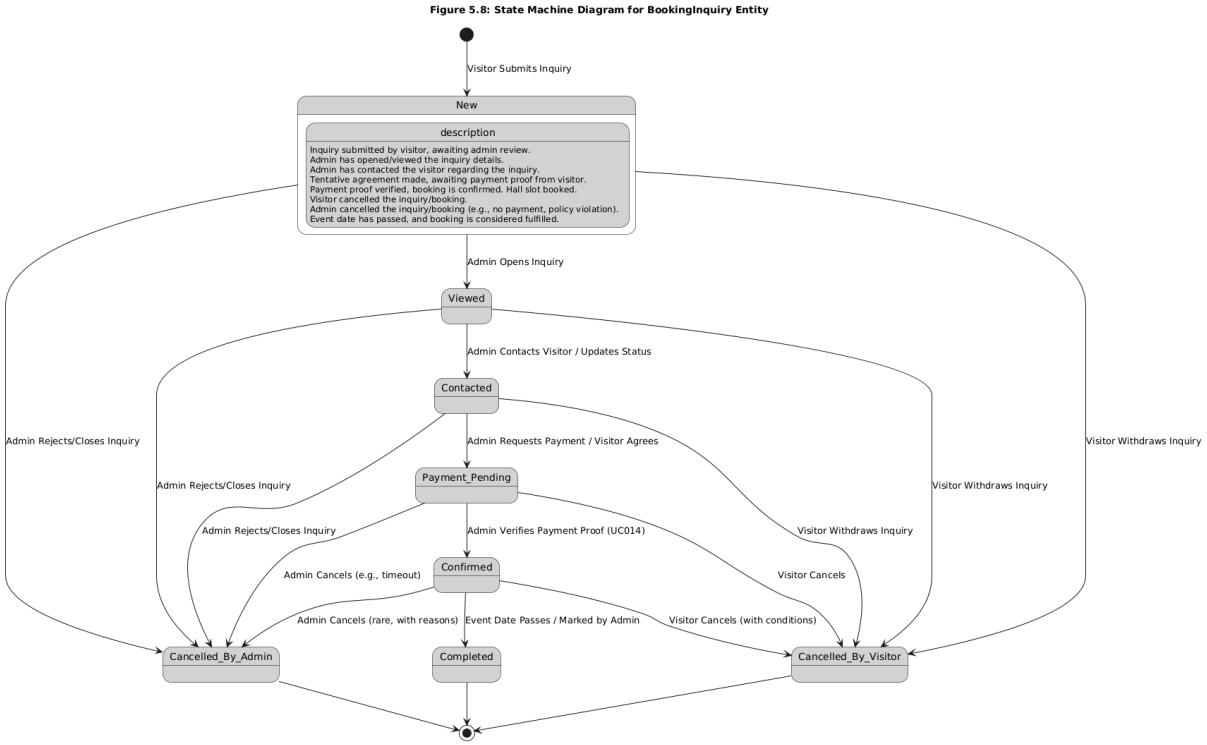


Figure 5.8: State Machine Diagram for BookingInquiry Entity

Description of BookingInquiry States:

The BookingInquiry entity transitions through several states from its creation to its resolution. It begins in a New state upon submission. An administrator action moves it to Viewed and then potentially Contacted. If discussions lead to a tentative agreement, it moves to Payment_Pending. Successful verification of payment (from the PaymentProof entity) transitions it to Confirmed. The inquiry can be Cancelled by either the visitor or admin at various stages. After the event date, a Confirmed booking moves to Completed.

Figure 5.9: State Machine Diagram for PaymentProof Entity

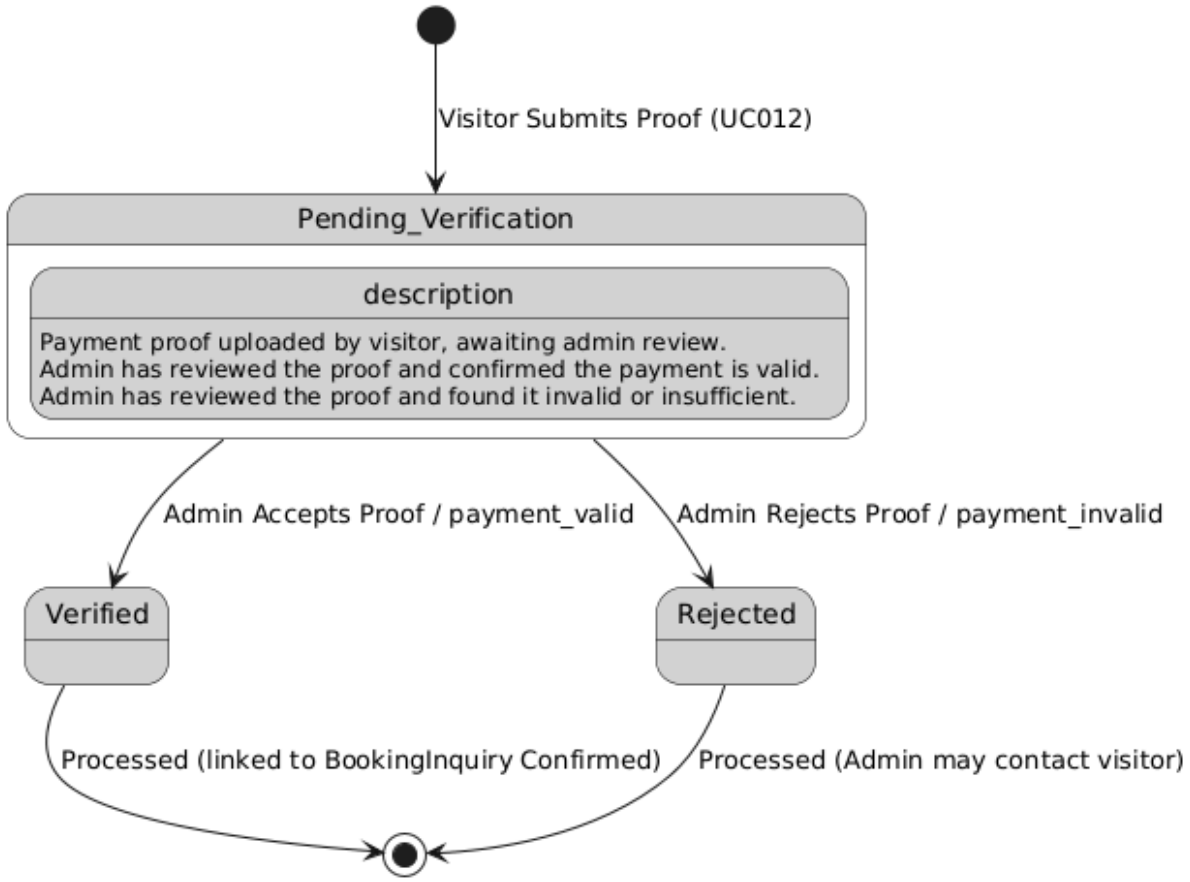


Figure 5.9: State Machine Diagram for PaymentProof Entity

Description of PaymentProof States:

The PaymentProof entity has a simpler lifecycle. Upon submission by a visitor, it enters the Pending_Verification state. The administrator then reviews it. If the proof is deemed valid, its state changes to Verified. If the proof is insufficient or invalid, its state changes to Rejected. Both Verified and Rejected are terminal states for the proof itself, triggering further actions related to the associated BookingInquiry.

These state machine diagrams illustrate the lifecycle and dynamic behavior of key entities within the Al-Muneer Online Portal, providing clarity on how they respond to system events and user interactions.

5.10 Algorithm Viewpoint

The Algorithm Viewpoint provides detailed design descriptions of operations, such as methods and functions, including the internal details and logic of each design entity. It provides details needed by programmers and analysts of algorithms in regard to time-space performance and processing logic prior to implementation, and to aid in producing unit test plans.

5.10.1 Design Concerns

The primary design concerns for the Algorithm Viewpoint are:

- **Processing Logic:** Detailing the step-by-step logic for specific operations or functions within the system.
- **Data Manipulation:** How data is transformed, calculated, or processed by algorithms.
- **Decision Points:** Clarifying conditions and branching logic within operations.
- **Clarity for Implementation:** Providing sufficient detail for developers to implement the algorithms correctly.

For the Al-Muneer Online Portal, the core processing logic for most functionalities aligns closely with the realization of the use cases.

5.10.2 Design View

The detailed step-by-step processing logic for the majority of user-facing and administrative operations in the Al-Muneer Online Portal is extensively documented through the Activity Diagrams provided for each Use Case (UC001 through UC016) in the Software Requirements Specification (SRS) document, Section 2.3: Use Case Details. These activity diagrams illustrate the flow of actions, decision points, and interactions required to fulfill each specific use case. They serve as the primary algorithmic design for those operations.

For example:

- The algorithm for a **Visitor Submitting a Booking Inquiry (UC005)** is detailed in its corresponding activity diagram in SRS Section 2.3.5. This includes steps for form access, data input, validation, saving the inquiry, and triggering notifications.
- The algorithm for an **Administrator Managing Payment Status (UC013)** is detailed in its activity diagram in SRS Section 2.3.14. This covers viewing proofs, offline verification, updating status in the system, and notifying the visitor.

Within the backend (Spring Boot application), this logic will primarily be implemented within the **Service layer classes** (e.g., InquiryServiceImpl, PaymentProofServiceImpl). These service methods will encapsulate the algorithms depicted in the SRS activity diagrams.

Key algorithmic considerations within these services include:

- **Data Validation:** All user inputs (from inquiry forms, admin content management, etc.) will undergo server-side validation logic as per defined business rules (e.g., mandatory fields, data formats, value ranges) before processing.
- **Availability Checking Logic:** When a visitor checks availability or an admin updates the calendar, the AvailabilityService will implement logic to query AvailabilitySlot entities for specific dates or date ranges and determine their status.
- **Pricing Calculation (if dynamic options are implemented):** If pricing involves selectable add-ons, the PricingService would contain logic to calculate total estimates based on selected base tiers and additional options.
- **Report Generation Logic:** The ReportService will contain logic to fetch data from various entities (e.g., BookingInquiry, Feedback) based on admin-specified parameters (like date ranges) and then aggregate/format this data into a simple, readable report structure for display. This will primarily involve structured database queries and data mapping.
- **File Handling for Payment Proofs:** The PaymentProofService (utilizing a FileUploadUtil) will implement the logic for receiving an uploaded file, validating its type and size, securely storing it in the designated file storage, and associating its path and metadata with the corresponding BookingInquiry.

The detailed design of these specific algorithms, where not fully elaborated by the SRS activity diagrams, will be straightforward implementations of the described business rules

within the respective Java service methods, leveraging Spring Boot and JPA functionalities for data access and transaction management. No overly complex or novel algorithms requiring specialized documentation beyond standard procedural logic are anticipated for the core features of this portal.

6. User Interface Design

This section describes the user interface design for the Al-Muneer Online Portal. It provides an overview of the UI from the user's perspective and conceptual layouts for key screens. The design aims for simplicity, usability, and a professional appearance appropriate for Al-Muneer Hall, with a primary focus on the Arabic language and cultural context of Yemen.

6.1 Overview of User Interface

The Al-Muneer Online Portal will feature a responsive web-based Graphical User Interface (GUI) accessible via standard desktop and mobile web browsers. The overall design philosophy emphasizes clarity, ease of navigation, and efficient task completion for both Visitors and the Administrator.

6.1.1 Visitor Interface:

Navigation will be managed through a clear, persistent main navigation menu, likely a header bar, offering access to key sections such as Home, About Us/Venue Info, Gallery, Availability, Pricing, Booking Inquiry, FAQ, Feedback, and Contact. Pages will generally adopt a clean layout, comprising a header, a main content area, and a footer that includes contact information and copyright details. Visuals, specifically images of the hall, will be incorporated judiciously to enhance the portal's appeal. The primary language for content and navigation will be Arabic, with potential for English equivalents for key navigation labels if deemed necessary. Interactivity will be facilitated through forms for inquiries, payment proof submission, and feedback. The availability calendar will be interactive, and image galleries will allow users to view larger versions of images. Users will receive clear confirmation messages for actions like inquiry submission or feedback, along with informative error messages.

6.1.2 Administrator Interface (Admin Panel):

Access to the admin panel will be secured via a login page. Navigation within the admin panel will be provided by a sidebar or top navigation menu, granting access to various management modules: Dashboard (for an overview), Content Management (covering Venue Info and FAQs), Media Gallery, Availability Calendar, Pricing, Inquiries, Payment Proofs, Feedback, Reports, and Notification Settings. The layout will typically feature a dashboard presenting summary information, followed by dedicated pages for managing specific data types, often employing tables for listing records and forms for editing or adding new ones. The primary language of the administrator interface will be Arabic to ensure ease of use for Mr. Almunajid. Interactivity in the admin panel will include data entry forms, tables with basic sorting and filtering capabilities, and action buttons such as Save, Update, Delete, View, Approve, and Reject. File upload widgets will also be present.

6.2 Screen Images (English Version)

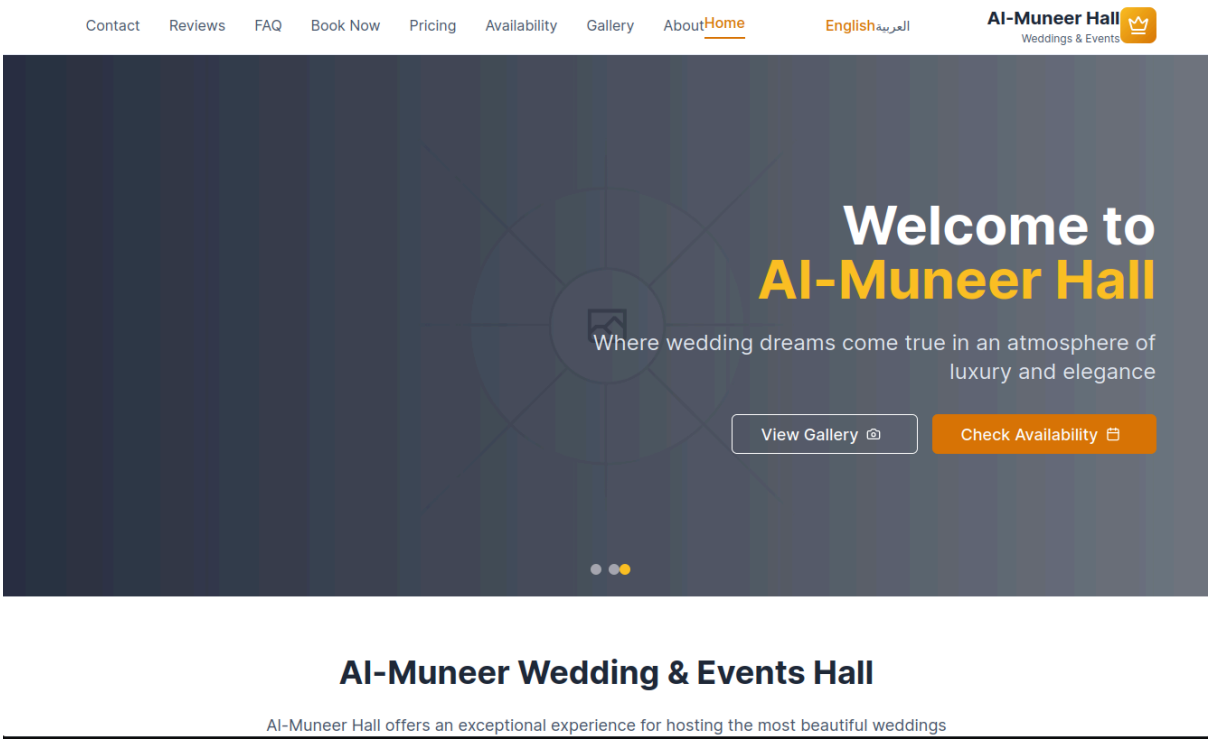


Figure 6.1: Visitor Interface - Homepage

This figure displays the homepage of the visitor interface, showcasing the initial impression and primary navigation elements available to users exploring the Al-Muneer Online Portal.

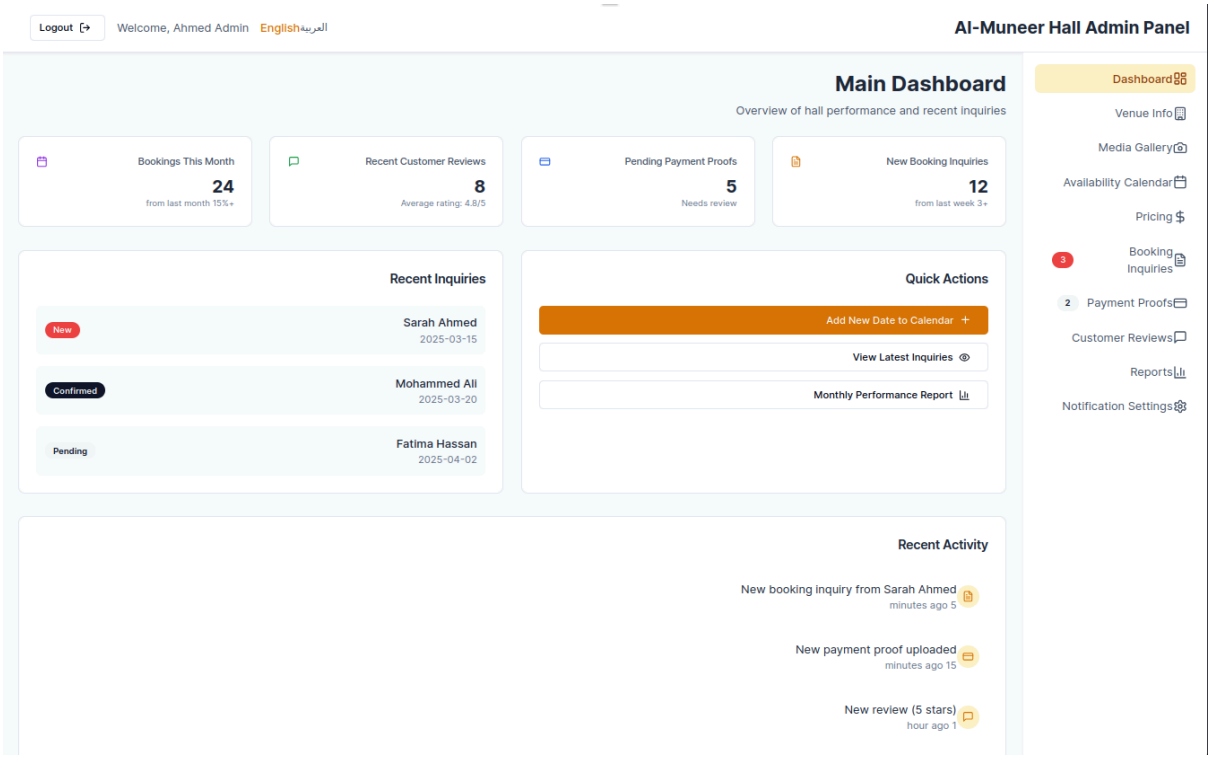


Figure 6.2: Administrator Interface - Admin Dashboard

This figure illustrates the administrator dashboard, providing an overview of key metrics and quick access to different management functions within the Al-Muneer Online Portal.

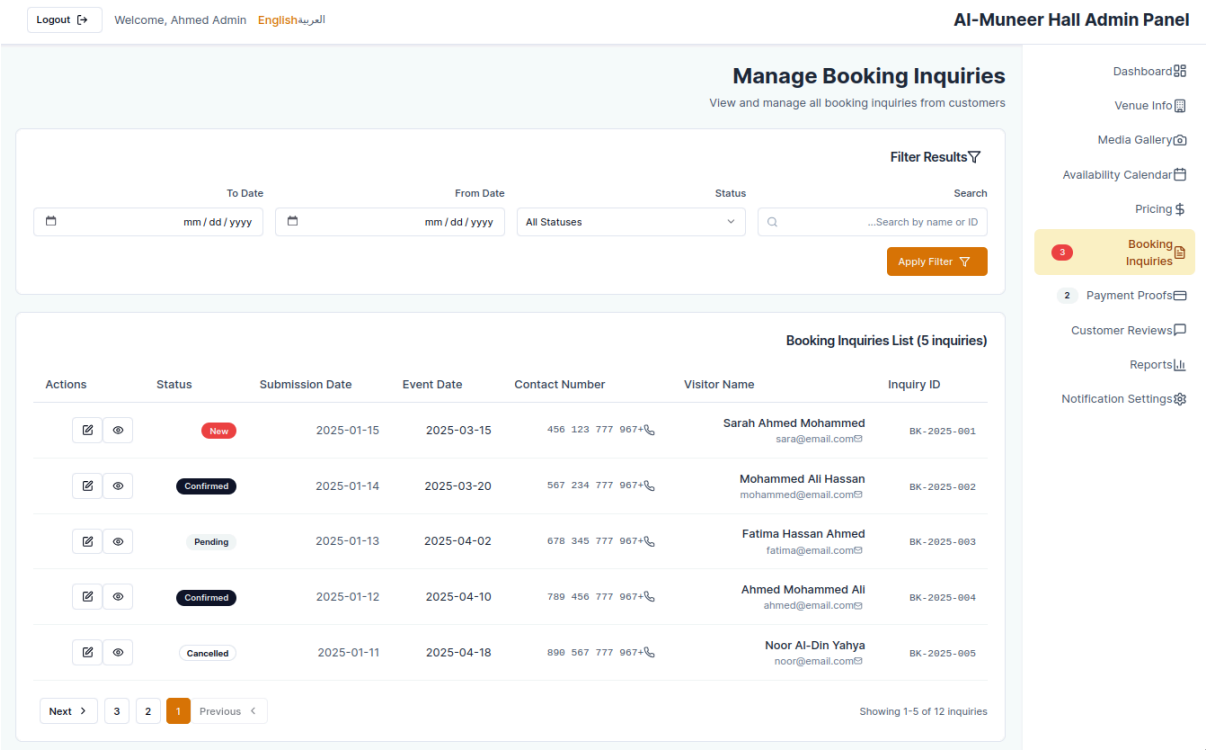


Figure 6.3: Administrator Interface - Manage Inquiries Page

This figure depicts the "Manage Inquiries" page within the administrator interface, showing how booking inquiries are listed and can be managed by the administrator.

7. Appendices

Appendix A: (Placeholder for Future Detailed API Specifications)

While the general interaction between frontend and backend is via RESTful APIs (as described in Section 5.6 Interface Viewpoint and implied in Section 5.8 Interaction Viewpoint), detailed API endpoint specifications (including exact request/response JSON structures, specific HTTP methods for all resources, and granular error codes) are typically developed iteratively during the implementation phase or documented in a separate API guide. If required for formal documentation beyond the scope of this initial SDD, they would be included here.

Appendix B: (Placeholder for Third-Party Service Integration Details)

If specific third-party services (e.g., a particular SMS gateway) are selected and their integration requires detailed configuration steps, API key management procedures, or specific interface contracts that are too extensive for the main design body, such details would be provided here. Currently, the design allows for such integrations, but the selection and detailed design of the interface with a specific provider is part of the implementation phase.